Published Work on Trend Filtering on Graphs and Its Relevance to `gflow`

gflow graph trend filtering notes

May 16, 2026

1 Short Answer

Yes: graph trend filtering exists, is published, and is directly relevant to the proposed `gflow` graph trend filtering project. The central paper is *Trend Filtering on Graphs* by Yu-Xiang Wang, James Sharpnack, Alex Smola, and Ryan Tibshirani. It appeared first around AISTATS 2015 and then in JMLR 2016.

That paper generalizes one-dimensional trend filtering to arbitrary graphs by penalizing the ℓ_1 norm of discrete graph differences. This is exactly the graph analogue of the mechanism that made one-dimensional trend filtering perform strongly in our path-graph benchmark: the method is locally adaptive, because an ℓ_1 penalty can set many graph differences exactly to zero while allowing selected nonsmooth changes.

2 The Core Estimator

Let $G = (V, E)$ be an undirected graph with $n = |V|$ vertices. A graph signal is a vector

$$\beta = (\beta_1, \ldots, \beta_n)^T \in \mathbb{R}^n,$$

where β_i is the fitted value at vertex i. Given noisy observations

$$y = (y_1, \ldots, y_n)^T,$$

graph trend filtering estimates β by solving

$$\widehat{\beta} = \underset{\beta \in \mathbb{R}^n}{\arg\min} \left\{ \frac{1}{2} \|y - \beta\|_2^2 + \lambda \|\Delta^{(k+1)} \beta\|_1 \right\}.$$

The two terms have different jobs:

- $\frac{1}{2}\|y - \beta\|_2^2$ keeps the fitted graph signal close to the observed response.
- $\lambda \|\Delta^{(k+1)} \beta\|_1$ penalizes graph roughness.

The key feature is the ℓ_1 norm. As in ordinary lasso, an ℓ_1 penalty encourages sparsity. But here the sparse quantities are not regression coefficients. They are graph differences of the fitted signal.

3 The $k = 0$ Case: Graph Fused Lasso

Choose an arbitrary orientation for every undirected edge. If an edge $e = (i, j)$ is oriented from i to j, the oriented incidence matrix D has one row per edge and one column per vertex:

$$D_{e,i} = -1, \qquad D_{e,j} = 1, \qquad D_{e,\ell} = 0 \quad (\ell \notin \{i, j\}).$$

Then

$$(D\beta)_e = \beta_j - \beta_i.$$

For $k = 0$, graph trend filtering uses

$$\Delta^{(1)} = D.$$

Therefore the penalty is

$$\|\Delta^{(1)}\beta\|_1 = \|D\beta\|_1 = \sum_{e=(i,j)\in E} |\beta_i - \beta_j|.$$

The optimization problem becomes

$$\widehat{\beta} = \underset{\beta\in\mathbb{R}^n}{\arg\min} \left\{ \frac{1}{2}\|y - \beta\|_2^2 + \lambda \sum_{(i,j)\in E} |\beta_i - \beta_j| \right\}.$$

This is graph fused lasso, also called graph total variation denoising.

Intuitively, the method encourages neighboring vertices to have equal fitted values, but it allows jumps on selected edges when the data strongly support them. Thus the fitted graph signal is often piecewise constant over graph regions.

4 Higher-Order Graph Trend Filtering

Let

$$L = D^T D$$

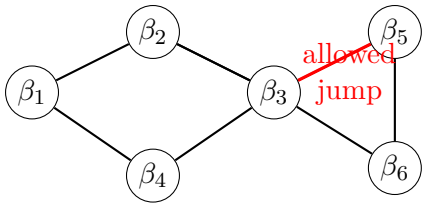
be the graph Laplacian. Wang, Sharpnack, Smola, and Tibshirani define higher-order graph difference operators by alternating incidence-like and Laplacian-like operations:

$$\Delta^{(k+1)} = \begin{cases} L^{(k+1)/2}, & k \text{ odd}, \\ DL^{k/2}, & k \text{ even}. \end{cases}$$

This gives a sequence of graph analogues of one-dimensional trend filtering:

Order k	Penalty	Interpretation
0	$\|D\beta\|_1$	edge jumps; piecewise constant signal
1	$\|L\beta\|_1$	graph Laplacian values; piecewise linear analogue
2	$\|DL\beta\|_1$	edge differences of graph Laplacian values; piecewise quadratic analogue

The words "piecewise linear" and "piecewise quadratic" are exact on a path graph and analogical on a general graph. On a general graph, there is no single left-to-right coordinate system. The operator nevertheless plays the same role: it measures higher-order graph roughness and uses an ℓ_1 penalty to make that roughness sparse.



$k = 0$: penalize $\sum_{(i,j)\in E} |\beta_i - \beta_j|$.
Most neighboring fitted values become equal or close; selected edges carry jumps.

Figure 1: Graph fused lasso, the $k = 0$ graph trend filtering case, encourages piecewise constant graph regions separated by a small number of active edges.

5 Visual Intuition

6 Why This Matters for `gflow`

The current metric graph low-pass smoother is an ℓ_2-spectral smoother. In its simplest unnormalized form, it is closely related to minimizing

$$\sum_i (y_i - f_i)^2 + \eta f^T L f.$$

This penalty smooths globally through the graph Laplacian. It suppresses high-frequency components, but the penalty is quadratic and therefore tends to spread smoothing everywhere.

Graph trend filtering would be the ℓ_1-adaptive counterpart:

$$\sum_i (y_i - \beta_i)^2 + \lambda \|\Delta^{(k+1)} \beta\|_1.$$

Because the penalty is ℓ_1, the fitted signal can be smooth or simple over most of the graph while preserving localized changes where the data support them.

This makes graph trend filtering a natural comparator to:

- `fit.metric.graph.lowpass()`, the metric-conductance graph low-pass smoother;
- `fit.rdgraph.regression()`, the existing Riemannian-complex/overlap-density graph smoother;
- one-dimensional `genlasso::trendfilter()`, when the graph is a path graph.

7 Weighted Graphs and Metric Conductances

The canonical graph trend filtering paper is often presented using an unweighted incidence matrix D. For a weighted graph, a natural extension is to use a weighted incidence operator. If $e = (i, j)$, one can define

$$(D_w \beta)_e = \omega_e (\beta_j - \beta_i).$$

Then the $k = 0$ penalty becomes

$$\|D_w \beta\|_1 = \sum_{e=(i,j)\in E} \omega_e |\beta_i - \beta_j|.$$

If the graph has conductances c_e, there are at least two plausible conventions:

$$(D_w\beta)_e = c_e(\beta_j - \beta_i)$$

or

$$(D_w\beta)_e = \sqrt{c_e}(\beta_j - \beta_i).$$

The square-root convention aligns with the quadratic Laplacian identity

$$\|D_w\beta\|_2^2 = \beta^T L_w \beta \quad \text{when} \quad L_w = D_w^T D_w.$$

The linear-conductance convention makes the ℓ_1 edge penalty scale directly as

$$\sum_e c_e|\beta_i - \beta_j|.$$

For `gflow`, this is a genuine modeling decision. Since `fit.metric.graph.lowpass()` already transforms metric edge lengths ℓ_e into conductances $c_e = \phi(\ell_e)$, a graph trend filtering comparator should probably expose the weighting rule explicitly rather than burying it inside the solver.

8 Related Published Work

Trend Filtering on Graphs

Yu-Xiang Wang, James Sharpnack, Alex Smola, and Ryan J. Tibshirani. *Trend Filtering on Graphs.* JMLR, 2016.

- arXiv: `https://arxiv.org/abs/1410.7690`
- JMLR PDF: `https://jmlr.csail.mit.edu/papers/volume17/15-147/15-147.pdf`
- Main contribution: graph trend filtering estimator and theory.

The DFS Fused Lasso

Oscar Hernan Madrid Padilla, James Sharpnack, James G. Scott, and Ryan J. Tibshirani. *The DFS Fused Lasso: Linear-Time Denoising over General Graphs.* JMLR, 2018.

- JMLR page: `https://www.jmlr.org/beta/papers/v18/16-532.html`
- Main contribution: scalable graph fused lasso / graph total variation denoising, mostly corresponding to the $k = 0$ case.

Fused ℓ_1 Trend Filtering on Graphs

Vladimir Pastukhov. *Fused ℓ_1 Trend Filtering on Graphs.* arXiv, 2024.

- arXiv: `https://arxiv.org/abs/2401.05250`
- Main contribution: fused graph trend filtering with additional fusion-style regularization and computationally feasible iterative solvers.

9 Suggested `gflow` Starting Point

The first implementation should probably start with weighted graph fused lasso:

$$\widehat{\beta} = \underset{\beta \in \mathbb{R}^n}{\arg\min} \left\{ \frac{1}{2} \|y - \beta\|_2^2 + \lambda \|D_w \beta\|_1 \right\}.$$

This is the $k = 0$ graph trend filtering case. It is easier to test and interpret than higher-order graph trend filtering, and it forces us to settle the key package questions:

- graph canonicalization;
- edge orientation;
- metric length to conductance transforms;
- weighted incidence convention;
- lambda grid and tuning behavior;
- return object structure.

Once that is stable, higher-order graph trend filtering can be introduced:

$$\widehat{\beta} = \underset{\beta \in \mathbb{R}^n}{\arg\min} \left\{ \frac{1}{2} \|y - \beta\|_2^2 + \lambda \|\Delta_w^{(k+1)} \beta\|_1 \right\}.$$